

Министерство образования Республики Беларусь
УО “Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «СПП»
Вариант 9

Выполнила:
студент 3 курса
группы ПО-12
Кравченко М.А.
Проверил:
Кулик А.Д.

Цель работы: приобрести практические навыки разработки API и баз данных.

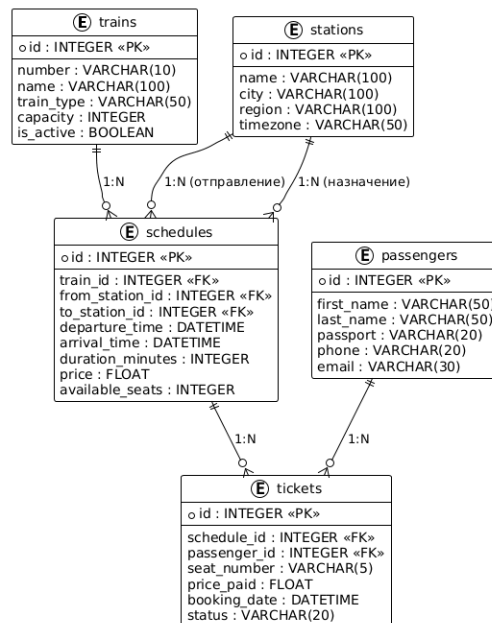
Задание.

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпоинт;

Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

Вариант 9. База данных Справочное бюро ж/д вокзала

Схема БД:



Код программы:

```
"""
База данных Справочное бюро ж/д вокзала.
FastAPI + SQLAlchemy + SQLite
"""

from datetime import datetime, date as date_type
from typing import Optional, List
```

```

from fastapi import FastAPI, HTTPException, Depends
from fastapi.responses import HTMLResponse
from pydantic import BaseModel, ConfigDict
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime, ForeignKey, Boolean
from sqlalchemy.orm import declarative_base, sessionmaker, Session, relationship

SQLALCHEMY_DATABASE_URL = "sqlite:///./railway_bureau.db"

engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
session_local = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

class Train(Base):
    """Модель поезда."""

    __tablename__ = "trains"

    id = Column(Integer, primary_key=True, index=True)
    number = Column(String(10), unique=True, nullable=False, index=True)
    name = Column(String(100))
    train_type = Column(String(50))
    capacity = Column(Integer)
    is_active = Column(Boolean, default=True)

    schedules = relationship("Schedule", back_populates="train")

    def get_info(self) -> dict:
        """Получить информацию о поезде."""
        return {
            "id": self.id,
            "number": self.number,
            "name": self.name,
            "train_type": self.train_type,
            "capacity": self.capacity,
            "is_active": self.is_active,
        }

    def deactivate(self) -> None:
        """Деактивировать поезд."""
        self.is_active = False

    def activate(self) -> None:
        """Активировать поезд."""
        self.is_active = True

class Station(Base):
    """Модель станции."""

    __tablename__ = "stations"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), unique=True, nullable=False, index=True)
    city = Column(String(100))
    region = Column(String(100))
    timezone = Column(String(50), default="Europe/Moscow")

    schedules_from = relationship(
        "Schedule",
        foreign_keys="Schedule.from_station_id",
        back_populates="from_station",
    )
    schedules_to = relationship(
        "Schedule",
        foreign_keys="Schedule.to_station_id",
        back_populates="to_station",
    )

```

```

)

def get_info(self) -> dict:
    """Получить информацию о станции."""
    return {
        "id": self.id,
        "name": self.name,
        "city": self.city,
        "region": self.region,
        "timezone": self.timezone,
    }

def get_full_name(self) -> str:
    """Получить полное название с городом."""
    return f'{self.name} ({self.city})'

class Schedule(Base):
    """Модель расписания."""

    __tablename__ = "schedules"

    id = Column(Integer, primary_key=True, index=True)
    train_id = Column(Integer, ForeignKey("trains.id"), nullable=False)
    from_station_id = Column(Integer, ForeignKey("stations.id"), nullable=False)
    to_station_id = Column(Integer, ForeignKey("stations.id"), nullable=False)
    departure_time = Column(DateTime, nullable=False)
    arrival_time = Column(DateTime, nullable=False)
    duration_minutes = Column(Integer)
    price = Column(Float, default=0)
    available_seats = Column(Integer, default=0)

    train = relationship("Train", back_populates="schedules")
    from_station = relationship("Station", foreign_keys=[from_station_id])
    to_station = relationship("Station", foreign_keys=[to_station_id])
    tickets = relationship("Ticket", back_populates="schedule")

    def get_info(self) -> dict:
        """Получить информацию о рейсе."""
        return {
            "id": self.id,
            "train_id": self.train_id,
            "from_station_id": self.from_station_id,
            "to_station_id": self.to_station_id,
            "departure_time": self.departure_time,
            "arrival_time": self.arrival_time,
            "duration_minutes": self.duration_minutes,
            "price": self.price,
            "available_seats": self.available_seats,
        }

    def has_free_seats(self) -> bool:
        """Проверить наличие свободных мест."""
        return self.available_seats > 0

class Passenger(Base):
    """Модель пассажира."""

    __tablename__ = "passengers"

    id = Column(Integer, primary_key=True, index=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    passport = Column(String(20), unique=True, nullable=False)
    phone = Column(String(20))
    email = Column(String(30))

```

```

tickets = relationship("Ticket", back_populates="passenger")

def get_info(self) -> dict:
    """Получить информацию о пассажире."""
    return {
        "id": self.id,
        "first_name": self.first_name,
        "last_name": self.last_name,
        "passport": self.passport,
        "phone": self.phone,
        "email": self.email,
    }

def get_full_name(self) -> str:
    """Получить полное имя пассажира."""
    return f"{self.last_name} {self.first_name}"

class Ticket(Base):
    """Модель билета."""

    __tablename__ = "tickets"

    id = Column(Integer, primary_key=True, index=True)
    schedule_id = Column(Integer, ForeignKey("schedules.id"), nullable=False)
    passenger_id = Column(Integer, ForeignKey("passengers.id"), nullable=False)
    seat_number = Column(String(5))
    price_paid = Column(Float, nullable=False)
    booking_date = Column(DateTime, default=datetime.now)
    status = Column(String(20), default="booked")

    schedule = relationship("Schedule", back_populates="tickets")
    passenger = relationship("Passenger", back_populates="tickets")

    def get_info(self) -> dict:
        """Получить информацию о билете."""
        return {
            "id": self.id,
            "schedule_id": self.schedule_id,
            "passenger_id": self.passenger_id,
            "seat_number": self.seat_number,
            "price_paid": self.price_paid,
            "booking_date": self.booking_date,
            "status": self.status,
        }

    def cancel(self) -> None:
        """Отменить билет."""
        self.status = "cancelled"

class TrainCreate(BaseModel):
    """Схема создания поезда."""

    number: str
    name: Optional[str] = None
    train_type: str
    capacity: int

class TrainResponse(TrainCreate):
    """Схема ответа поезда."""

    model_config = ConfigDict(from_attributes=True)
    id: int
    is_active: bool

```

```

class StationCreate(BaseModel):
    """Схема создания станции."""

    name: str
    city: str
    region: Optional[str] = None
    timezone: Optional[str] = "Europe/Moscow"

class StationResponse(StationCreate):
    """Схема ответа станции."""

    model_config = ConfigDict(from_attributes=True)
    id: int

class ScheduleCreate(BaseModel):
    """Схема создания расписания."""

    train_id: int
    from_station_id: int
    to_station_id: int
    departure_time: datetime
    arrival_time: datetime
    price: float
    available_seats: int

class ScheduleResponse(ScheduleCreate):
    """Схема ответа расписания."""

    model_config = ConfigDict(from_attributes=True)
    id: int
    duration_minutes: Optional[int] = None

class PassengerCreate(BaseModel):
    """Схема создания пассажира."""

    first_name: str
    last_name: str
    passport: str
    phone: Optional[str] = None
    email: Optional[str] = None

class PassengerResponse(PassengerCreate):
    """Схема ответа пассажира."""

    model_config = ConfigDict(from_attributes=True)
    id: int

class TicketCreate(BaseModel):
    """Схема создания билета."""

    schedule_id: int
    passenger_id: int
    seat_number: Optional[str] = None

class TicketResponse(TicketCreate):
    """Схема ответа билета."""

    model_config = ConfigDict(from_attributes=True)
    id: int
    price_paid: float
    booking_date: datetime

```

```

status: str

def get_db():
    """Получить сессию БД."""
    db = session_local()
    try:
        yield db
    finally:
        db.close()

app = FastAPI(title="Справочное бюро ж/д вокзала")

@app.get("/", response_class=HTMLResponse)
def root():
    """Главная страница со списком эндпоинтов."""
    return """
<html>
<head><title>Ж/Д Справочное бюро API</title></head>
<body>
<h1>Справочное бюро ж/д вокзала API</h1>
<h2>Эндпоинты:</h2>
<ul>
<li><b>Поезда</b>
<ul>
<li>GET /trains - список</li>
<li>POST /trains - добавить</li>
<li>PUT /trains/{id} - обновить</li>
<li>DELETE /trains/{id} - удалить</li>
</ul>
</li>
<li><b>Станции</b>
<ul>
<li>GET /stations - список</li>
<li>POST /stations - добавить</li>
<li>DELETE /stations/{id} - удалить</li>
</ul>
</li>
<li><b>Расписание</b>
<ul>
<li>GET /schedules - все</li>
<li>GET /schedules/search - поиск</li>
<li>POST /schedules - добавить</li>
</ul>
</li>
<li><b>Пассажиры</b>
<ul>
<li>GET /passengers - список</li>
<li>POST /passengers - добавить</li>
<li>PUT /passengers/{id} - обновить</li>
<li>DELETE /passengers/{id} - удалить</li>
</ul>
</li>
<li><b>Билеты</b>
<ul>
<li>GET /tickets - все</li>
<li>POST /tickets - купить</li>
<li>DELETE /tickets/{id} - отменить</li>
</ul>
</li>
</ul>
</body>
</html>
"""

```

```

@app.get("/trains", response_model=List[TrainResponse])
def get_trains(db: Session = Depends(get_db)):
    """Получить список всех поездов."""
    return db.query(Train).filter(Train.is_active.is_(True)).all()

@app.post("/trains", response_model=TrainResponse)
def create_train(train: TrainCreate, db: Session = Depends(get_db)):
    """Добавить новый поезд."""
    db_train = Train(**train.model_dump())
    db.add(db_train)
    db.commit()
    db.refresh(db_train)
    return db_train

@app.put("/trains/{train_id}", response_model=TrainResponse)
def update_train(train_id: int, train: TrainCreate, db: Session = Depends(get_db)):
    """Обновить данные поезда."""
    db_train = db.query(Train).filter(Train.id == train_id).first()
    if not db_train:
        raise HTTPException(status_code=404, detail="Поезд не найден")
    for key, value in train.model_dump().items():
        setattr(db_train, key, value)
    db.commit()
    db.refresh(db_train)
    return db_train

@app.delete("/trains/{train_id}")
def delete_train(train_id: int, db: Session = Depends(get_db)):
    """Удалить поезд."""
    train = db.query(Train).filter(Train.id == train_id).first()
    if not train:
        raise HTTPException(status_code=404, detail="Поезд не найден")
    train.is_active = False
    db.commit()
    return {"message": "Поезд удален"}

@app.get("/stations", response_model=List[StationResponse])
def get_stations(db: Session = Depends(get_db)):
    """Получить список всех станций."""
    return db.query(Station).all()

@app.post("/stations", response_model=StationResponse)
def create_station(station: StationCreate, db: Session = Depends(get_db)):
    """Добавить новую станцию."""
    db_station = Station(**station.model_dump())
    db.add(db_station)
    db.commit()
    db.refresh(db_station)
    return db_station

@app.delete("/stations/{station_id}")
def delete_station(station_id: int, db: Session = Depends(get_db)):
    """Удалить станцию."""
    station = db.query(Station).filter(Station.id == station_id).first()
    if not station:
        raise HTTPException(status_code=404, detail="Станция не найдена")
    db.delete(station)
    db.commit()
    return {"message": "Станция удалена"}

@app.get("/schedules", response_model=List[ScheduleResponse])

```



```

def get_schedules(db: Session = Depends(get_db)):
    """Получить все рейсы."""
    return db.query(Schedule).all()

@app.get("/schedules/search")
def search_schedules(
    from_station: str,
    to_station: str,
    search_date: date_type,
    db: Session = Depends(get_db),
):
    """Поиск рейсов по станциям и дате."""
    start_time = datetime.combine(search_date, datetime.min.time())
    end_time = datetime.combine(search_date, datetime.max.time())

    results = (
        db.query(Schedule)
        .join(Station, Schedule.from_station_id == Station.id)
        .join(Station, Schedule.to_station_id == Station.id)
        .filter(
            Station.name == from_station,
            Station.name == to_station,
            Schedule.departure_time.between(start_time, end_time),
        )
        .all()
    )

    return [
        {
            "id": s.id,
            "train_number": s.train.number,
            "train_name": s.train.name,
            "from_station": s.from_station.name,
            "to_station": s.to_station.name,
            "departure_time": s.departure_time,
            "arrival_time": s.arrival_time,
            "price": s.price,
            "available_seats": s.available_seats,
        }
        for s in results
    ]

@app.post("/schedules", response_model=ScheduleResponse)
def create_schedule(schedule: ScheduleCreate, db: Session = Depends(get_db)):
    """Добавить новый рейс."""
    duration = int((schedule.arrival_time - schedule.departure_time).total_seconds() / 60)
    db_schedule = Schedule(**schedule.model_dump(), duration_minutes=duration)
    db.add(db_schedule)
    db.commit()
    db.refresh(db_schedule)
    return db_schedule

@app.get("/passengers", response_model=List[PassengerResponse])
def get_passengers(db: Session = Depends(get_db)):
    """Получить список всех пассажиров."""
    return db.query(Passenger).all()

@app.post("/passengers", response_model=PassengerResponse)
def create_passenger(passenger: PassengerCreate, db: Session = Depends(get_db)):
    """Добавить нового пассажира."""
    db_passenger = Passenger(**passenger.model_dump())
    db.add(db_passenger)
    db.commit()
    db.refresh(db_passenger)

```

```

return db_passenger

@app.put("/passengers/{passenger_id}", response_model=PassengerResponse)
def update_passenger(passenger_id: int, passenger: PassengerCreate, db: Session = Depends(get_db)):
    """Обновить данные пассажира."""
    db_passenger = db.query(Passenger).filter(Passenger.id == passenger_id).first()
    if not db_passenger:
        raise HTTPException(status_code=404, detail="Пассажир не найден")
    for key, value in passenger.model_dump().items():
        setattr(db_passenger, key, value)
    db.commit()
    db.refresh(db_passenger)
    return db_passenger

@app.delete("/passengers/{passenger_id}")
def delete_passenger(passenger_id: int, db: Session = Depends(get_db)):
    """Удалить пассажира."""
    passenger = db.query(Passenger).filter(Passenger.id == passenger_id).first()
    if not passenger:
        raise HTTPException(status_code=404, detail="Пассажир не найден")
    db.delete(passenger)
    db.commit()
    return {"message": "Пассажир удален"}

@app.get("/tickets", response_model=List[TicketResponse])
def get_tickets(db: Session = Depends(get_db)):
    """Получить все билеты."""
    return db.query(Ticket).all()

@app.post("/tickets", response_model=TicketResponse)
def create_ticket(ticket: TicketCreate, db: Session = Depends(get_db)):
    """Купить билет."""
    schedule = db.query(Schedule).filter(Schedule.id == ticket.schedule_id).first()
    if not schedule:
        raise HTTPException(status_code=404, detail="Рейс не найден")
    if not schedule.has_free_seats():
        raise HTTPException(status_code=400, detail="Нет свободных мест")

    db_ticket = Ticket(
        schedule_id=ticket.schedule_id,
        passenger_id=ticket.passenger_id,
        seat_number=ticket.seat_number,
        price_paid=schedule.price,
        status="paid",
    )
    schedule.available_seats -= 1
    db.add(db_ticket)
    db.commit()
    db.refresh(db_ticket)
    return db_ticket

@app.delete("/tickets/{ticket_id}")
def cancel_ticket(ticket_id: int, db: Session = Depends(get_db)):
    """Отменить билет."""
    ticket = db.query(Ticket).filter(Ticket.id == ticket_id).first()
    if not ticket:
        raise HTTPException(status_code=404, detail="Билет не найден")
    schedule = db.query(Schedule).filter(Schedule.id == ticket.schedule_id).first()
    if schedule:
        schedule.available_seats += 1
    ticket.cancel()
    db.commit()
    return {"message": "Билет отменен"}

```

```

def _create_stations(db: Session) -> None:
    """Создать тестовые станции."""
    stations_data = [
        ("Москва", "Москва", "Московская область"),
        ("Санкт-Петербург", "Санкт-Петербург", "Ленинградская область"),
        ("Казань", "Казань", "Татарстан"),
        ("Екатеринбург", "Екатеринбург", "Свердловская область"),
    ]
    for name, city, region in stations_data:
        db.add(Station(name=name, city=city, region=region))
    db.commit()

def _create_trains(db: Session) -> None:
    """Создать тестовые поезда."""
    trains_data = [
        ("001А", "Красная стрела", "скорый", 500),
        ("002Б", "Сапсан", "скоростной", 600),
        ("003В", None, "пассажирский", 400),
    ]
    for number, name, ttype, capacity in trains_data:
        db.add(Train(number=number, name=name, train_type=ttype, capacity=capacity))
    db.commit()

def _create_schedules(db: Session) -> None:
    """Создать тестовое расписание."""
    msk = db.query(Station).filter(Station.name == "Москва").first()
    spb = db.query(Station).filter(Station.name == "Санкт-Петербург").first()
    kzn = db.query(Station).filter(Station.name == "Казань").first()
    train1 = db.query(Train).filter(Train.number == "001А").first()
    train2 = db.query(Train).filter(Train.number == "002Б").first()

    # fmt: off
    schedules_data = [
        (train1.id, msk.id, spb.id,
         datetime(2026, 5, 20, 8, 0),
         datetime(2026, 5, 20, 15, 30), 3500, 100),
        (train2.id, msk.id, kzn.id,
         datetime(2026, 5, 20, 10, 0),
         datetime(2026, 5, 20, 18, 0), 2800, 80),
    ]
    # fmt: on

    for tid, fid, t2id, dep, arr, price, seats in schedules_data:
        duration = int((arr - dep).total_seconds() / 60)
        db.add(
            Schedule(
                train_id=tid,
                from_station_id=fid,
                to_station_id=t2id,
                departure_time=dep,
                arrival_time=arr,
                duration_minutes=duration,
                price=price,
                available_seats=seats,
            )
        )
    db.commit()

def _create_passengers(db: Session) -> None:
    """Создать тестовых пассажиров."""
    passengers_data = [
        ("Иван", "Петров", "1234567890", "+7-123-456-78-90", "ivan@mail.ru"),
        ("Анна", "Сидорова", "2345678901", "+7-234-567-89-01", "anna@mail.ru"),
    ]

```

```

]
for fname, lname, passport, phone, email in passengers_data:
    db.add(
        Passenger(
            first_name=fname,
            last_name=lname,
            passport=passport,
            phone=phone,
            email=email,
        )
    )
db.commit()

def init_db():
    """Инициализация БД с тестовыми данными."""
    Base.metadata.create_all(bind=engine)
    db = session_local()

    if db.query(Train).count() > 0:
        db.close()
        return

    _create_stations(db)
    _create_trains(db)
    _create_schedules(db)
    _create_passengers(db)

    db.close()
    print("База данных инициализирована")

if __name__ == "__main__":
    init_db()
    import uvicorn

    uvicorn.run("database:app", host="127.0.0.1", port=8000, reload=True)

```

The screenshot displays a REST client interface for a POST request to the endpoint `/trains` with the action 'Create Train'. The 'Parameters' section is empty, and the 'Request body' is marked as required. The 'Edit Value' section shows the JSON payload for the request, which is a list of four train objects. The 'Response body' section shows the JSON response, which is a list of the same four train objects, each now assigned a unique ID (1, 2, 3, and 4 respectively).

Request body:

```

[
  {
    "number": "001A",
    "name": "Красная стрела",
    "train_type": "скорый",
    "capacity": 500,
    "id": 1,
    "is_active": true
  },
  {
    "number": "002B",
    "name": "Самсан",
    "train_type": "скоростной",
    "capacity": 600,
    "id": 2,
    "is_active": true
  },
  {
    "number": "003B",
    "name": null,
    "train_type": "пассажирский",
    "capacity": 400,
    "id": 3,
    "is_active": true
  }
]

```

Response body:

```

[
  {
    "id": 1,
    "is_active": true
  },
  {
    "number": "002B",
    "name": "Самсан",
    "train_type": "скоростной",
    "capacity": 600,
    "id": 2,
    "is_active": true
  },
  {
    "number": "003B",
    "name": null,
    "train_type": "пассажирский",
    "capacity": 400,
    "id": 3,
    "is_active": true
  },
  {
    "number": "004D",
    "name": "ОксанаКрутая",
    "train_type": "плацкарт",
    "capacity": 300,
    "id": 4,
    "is_active": true
  }
]

```

Вывод: приобрела практические навыки разработки API и баз данных.